

Sequential Task:

Threads - synchronization barrier

Object-Oriented Programming and GUI

May 11, 2026

1. Declare a class `CustomBarrier`, which implements a synchronization barrier mechanism for a specified number of threads.
 - (a) In the class body, include the following fields:
 - add a private immutable field `limit` of type `int`, which specifies the number of threads required to reach the barrier.
 - add a private field `count` of type `int`, which stores the current number of threads waiting at the barrier.
 - Initialize the field with the value 0.
 - add a private field `generation` of type `int`, which identifies the current barrier generation.
 - Initialize the field with the value 0.
 - (b) Implement the constructor `CustomBarrier(int limit)`.
 - i. Validate the correctness of the `limit` parameter.
 - If `limit <= 0`, throw an `IllegalArgumentException` with an appropriate message, *check the program output*.
 - ii. Assign the parameter value to the `limit` field.
 - (c) Implement the `await()` method, which synchronizes threads at the barrier.
 - i. Mark the method as `synchronized` to ensure mutual exclusion.
 - ii. Store the current value of the `generation` field in the local variable `currentGeneration`.
 - iii. Increase the waiting thread counter by incrementing the `count` field.
 - iv. Display a message containing the name of the current thread, the current number of waiting threads, and the generation identifier.
 - v. Check whether the number of waiting threads is less than `limit`.
 - If so, put the thread into the waiting state.
 - Use a `while` loop to wait as long as `currentGeneration == generation` and `count < limit`.
 - Call the `wait()` method to suspend the thread.
 - vi. Otherwise, handle the situation in which the current thread is the last one required to reach the barrier.

- Display a message informing that the barrier has been reached.
 - Reset the `count` counter to 0.
 - Increase the value of `generation` to mark the beginning of a new phase.
 - Call the `notifyAll()` method to resume all waiting threads.
- (d) Implement the `getWaitingCount()` method.
- i. Mark the method as `synchronized`.
 - ii. Return the current value of the `count` field.
- (e) Implement the `getGeneration()` method.
- i. Mark the method as `synchronized`.
 - ii. Return the current value of the `generation` field.
- (f) Implement the `reset()` method.
- i. Mark the method as `synchronized`.
 - ii. Set the `count` field to 0.
 - iii. Increase the value of the `generation` field.
 - iv. Call the `notifyAll()` method to resume all waiting threads.
2. Declare a class `Worker`, which extends the `Thread` class and represents a thread performing a task in stages synchronized by `CustomBarrier`.
- (a) In the class body, include the following fields:
- i. Add a private immutable field `barrier` of type `CustomBarrier`, which stores a reference to the synchronization barrier.
 - ii. Add a private immutable field `taskDuration` of type `int`, which defines the duration of a single task phase.
 - iii. Add a private immutable field `taskName` of type `String`, which stores the name of the task being performed.
- (b) Implement the constructor `Worker(String name, CustomBarrier barrier, int taskDuration, String taskName)`.
- i. Call the superclass constructor `Thread`, passing the `name` parameter as the thread name.
 - ii. Assign the passed barrier to the `barrier` field.
 - iii. Assign the passed task duration to the `taskDuration` field.
 - iv. Assign the passed task name to the `taskName` field.
- (c) Override the `run()` method, which defines the full course of the thread's work.
- i. Place the phase execution logic inside a `try` block to handle possible thread interruption.
 - ii. Execute the first work phase by calling `executeTask(1)`.
 - iii. After completing the first phase, call `barrier.await()` to stop the thread until the remaining threads reach the barrier.

- iv. Execute the second work phase by calling `executeTask(2)`.
 - v. After completing the second phase, call `barrier.await()` again.
 - vi. Execute the third work phase by calling `executeTask(3)`.
 - vii. After completing the third phase, call `barrier.await()` again.
 - viii. Display a message informing that the current thread has completed all phases.
 - ix. Handle the `InterruptedException`.
 - Display an error message informing that the current thread has been interrupted.
 - Restore the thread interruption status by calling `Thread.currentThread().interrupt()`.
- (d) Implement the private method `executeTask(int phase)`, whose task is to execute a single task phase.
- i. Accept the parameter `phase`, which specifies the number of the executed phase.
 - ii. Declare that the method may throw an `InterruptedException`.
 - iii. Display a message informing that the current thread is starting the specified phase and provide the task name `taskName`.
 - iv. Pause the thread execution for the time stored in the `taskDuration` field, using `Thread.sleep(taskDuration)`.
 - v. Display a message informing that the current thread has completed the specified phase.

3. Check whether the program output is:

```

=== SYNCHRONIZATION BARRIER DEMO ===

Worker-3 starting phase 1 (Validation)
Worker-2 starting phase 1 (Processing)
Worker-4 starting phase 1 (Analysis)
Worker-1 starting phase 1 (Loading data)
Worker-5 starting phase 1 (Reporting)
Worker-1 completed phase 1
Worker-1 arrived at barrier (1/5) [generation: 0]
Worker-4 completed phase 1
Worker-4 arrived at barrier (2/5) [generation: 0]
Worker-2 completed phase 1
Worker-2 arrived at barrier (3/5) [generation: 0]
Worker-5 completed phase 1
Worker-5 arrived at barrier (4/5) [generation: 0]
Worker-3 completed phase 1
Worker-3 arrived at barrier (5/5) [generation: 0]
>>> Barrier reached! Releasing all threads <<<
Worker-3 starting phase 2 (Validation)
Worker-4 starting phase 2 (Analysis)
Worker-1 starting phase 2 (Loading data)
Worker-5 starting phase 2 (Reporting)
Worker-2 starting phase 2 (Processing)

```

```

Worker-1 completed phase 2
Worker-1 arrived at barrier (1/5) [generation: 1]
Worker-4 completed phase 2
Worker-4 arrived at barrier (2/5) [generation: 1]
Worker-2 completed phase 2
Worker-2 arrived at barrier (3/5) [generation: 1]
Worker-5 completed phase 2
Worker-5 arrived at barrier (4/5) [generation: 1]
Worker-3 completed phase 2
Worker-3 arrived at barrier (5/5) [generation: 1]
>>> Barrier reached! Releasing all threads <<<
Worker-3 starting phase 3 (Validation)
Worker-4 starting phase 3 (Analysis)
Worker-1 starting phase 3 (Loading data)
Worker-5 starting phase 3 (Reporting)
Worker-2 starting phase 3 (Processing)
Worker-1 completed phase 3
Worker-1 arrived at barrier (1/5) [generation: 2]
Worker-4 completed phase 3
Worker-4 arrived at barrier (2/5) [generation: 2]
Worker-2 completed phase 3
Worker-2 arrived at barrier (3/5) [generation: 2]
Worker-5 completed phase 3
Worker-5 arrived at barrier (4/5) [generation: 2]
Worker-3 completed phase 3
Worker-3 arrived at barrier (5/5) [generation: 2]
>>> Barrier reached! Releasing all threads <<<
Worker-3 completed all phases!
Worker-2 completed all phases!
Worker-4 completed all phases!
Worker-1 completed all phases!
Worker-5 completed all phases!

```

=== ALL THREADS COMPLETED ===

4. ★ Milestone 1: present the result to the instructor